

1. Project Overview

- **Project:** A Command-Line Interface (CLI) Flight Management System.
 - **Language:** C++
 - **Core Principle:** Must be built using Object-Oriented Programming (OOP) principles (encapsulation, inheritance, polymorphism, abstraction).
 - **Primary Goal:** To allow users (passengers) to search for and book flights, and for an administrator to manage flight and booking data.
-

2. Object-Oriented Design Requirements

Your system's design should be built around these core classes. This is not an exhaustive list, but it's the foundation for your OOP structure.

- **Flight Class:**
 - **Attributes:** `flightNumber` (string), `origin` (string), `destination` (string), `departureTime` (datetime), `arrivalTime` (datetime), `totalSeats` (int), `availableSeats` (int), `price` (double).
 - **Methods:** `displayDetails()`, `bookSeat()`, `cancelSeat()`, `isFull()`, `getAvailableSeats()`.
- **Passenger Class:**
 - **Attributes:** `passengerID` (string/int), `firstName` (string), `lastName` (string), `email` (string), `contactNumber` (string).
 - **Methods:** `displayDetails()`, `updateContactInfo()`.
- **Booking / Reservation Class:**
 - **Attributes:** `bookingID` (string/int), `passenger` (Passenger object), `flight` (Flight object), `seatNumber` (string), `bookingStatus` (e.g., "Confirmed", "Cancelled").
 - **Methods:** `confirmBooking()`, `cancelBooking()`, `displayBookingDetails()`.
- **FlightDatabase / SystemManager Class (Singleton Pattern recommended):**
 - **Attributes:** `allFlights` (e.g., `std::vector<Flight>`), `allBookings` (e.g., `std::vector<Booking>`).
 - **Methods:** `addFlight()`, `removeFlight()`, `findFlightByID()`, `searchFlights(origin, destination, date)`, `generateBookingID()`, `saveDataToFile()`, `loadDataFromFile()`.
- **User Class (Optional but recommended):**

- A base class for **Passenger** and **Admin** if you implement user accounts.
 - **Attributes:** `username`, `password`.
 - **Methods:** `login()`, `logout()`.
 - **Admin Class (Optional, could inherit from User):**
 - **Methods:** `adminAddFlight()`, `adminRemoveFlight()`, `viewAllBookings()`.
-

3. Functional Requirements

These are the features the system **must** perform, divided into user roles.

User (Passenger) Functions

- **Search Flights:** The user must be able to search for available flights based on:
 - Origin
 - Destination
 - Date
- **View Flight Details:** After searching, the user must be able to see a list of matching flights, including flight number, times, price, and available seats.
- **Book a Flight:** The user must be able to select a flight and provide passenger details (First Name, Last Name, etc.) to make a booking.
 - The system must confirm the booking and generate a unique Booking ID.
 - `availableSeats` for the `Flight` must decrease by one.
- **Cancel a Booking:** The user must be able to cancel an existing booking using their Booking ID.
 - The system must confirm the cancellation.
 - `availableSeats` for the corresponding `Flight` must increase by one.
- **View My Bookings:** The user should be able to retrieve and display details of their bookings (e.g., by providing their name or a passenger ID).

Administrator Functions

- **Admin Login:** The system should have a simple, separate "admin mode" (e.g., by entering a specific password).
- **Add New Flight:** The admin must be able to add new flights to the system by providing all necessary details (flight number, origin, destination, times, total seats).
- **Remove Flight:** The admin must be able to remove a flight from the schedule (e.g., by entering the `flightNumber`).
- **View All Flights:** The admin must be able to list all flights currently in the system.

- **View All Bookings:** The admin must be able to see a complete list of all bookings for all flights.
-

4. Non-Functional Requirements

These define the *quality* and *constraints* of your CLI application.

- **Usability (CLI):**
 - The CLI menu must be clear, intuitive, and easy to navigate.
 - All prompts for user input (e.g., "Enter origin:") must be unambiguous.
 - The system must provide clear confirmation messages (e.g., "Booking confirmed! ID: 12345").
- **Error Handling:**
 - The system must handle invalid inputs gracefully (e.g., "Invalid option, please try again.").
 - It must prevent impossible actions (e.g., booking a flight that is full, booking a flight that doesn't exist).
 - Input validation is required (e.g., ensure a seat number is not a random string).
- **Persistence:**
 - The system must **save** all flight and booking data to a file (e.g., `.txt` or `.csv`) when the program exits.
 - The system must **load** all data from the file when the program starts. This ensures data isn't lost.
- **Performance:**
 - All CLI commands (searching, booking) should complete almost instantly (e.g., < 2 seconds).